TP 1 — Sécuriser le stockage d'un jeton de session

Module Sécurité des Smartphones · Séance 5 · Chaymae Noughou
Outils : Android Studio, Kotlin, émulateur Pixel 8 (Android 15, API 35), ADB · Application de laboratoire **MobileVault** (`edu.lab.mobilevault`) · Données entièrement fictives

Introduction : le problème et la démarche

Une application mobile garde souvent un **jeton de session** pour que l'utilisateur reste connecté. Si ce jeton est volé, l'attaquant peut se faire passer pour l'utilisateur. L'application étudiée présente plusieurs vulnérabilités. Elle :

- stocke le jeton et une note privée **en clair** dans ses préférences (SharedPreferences) ;
- écrit le jeton dans les **journaux système** (Logcat) ;
- contient un **mot de passe écrit en dur** dans le code et présente **Base64** comme une « protection ».

La démarche suivie est celle d'un ingénieur sécurité : **reproduire la faille → corriger → vérifier la correction (contre-test) → identifier les limites**.

> **Lien avec le cours :** encodage ≠ hachage ≠ chiffrement (3.1) · AES-GCM, nonce et tag (3.3–3.4) · Android Keystore (chapitre 4) · séparation des responsabilités dans le code (chapitre 5).

Vulnérabilités identifiées et corrections

L'audit du code et les tests ont permis d'identifier **7 vulnérabilités** dans l'application d'origine : un secret embarqué, des données sensibles en clair, un booléen de session, Base64 utilisé comme « protection », un log sensible et une sauvegarde permissive. Pour chacune, ce rapport présente le code d'origine, pourquoi c'est dangereux, la correction appliquée et l'endroit où elle est prouvée.

E1 — Secret (mot de passe) écrit en dur dans le code

📍 MainActivity.kt, ligne 13

```
private const val LAB_PASSWORD = "vault-demo-2026"
...
if (password.text.toString() == LAB_PASSWORD) { ... }
```

❌ **Pourquoi c'est dangereux :** Le mot de passe est dans l'APK : n'importe qui peut le lire en décompilant l'application (jadx, apktool). Le contrôle d'identité est fait **sur le téléphone**, donc il peut être contourné.

✅ **Correction :** La version corrigée de `MainActivity.kt` ne contient plus aucun mot de passe : la connexion est fictive. Dans une vraie application, l'identité est vérifiée **par le serveur**, et aucun secret partagé n'est embarqué dans l'APK.

```
addButton("1. Connexion fictive") { store.save("demo_" + UUID.randomUUID().toString()) ... }
```

Preuve dans ce rapport : Étape 1 (découverte dans le code), étape 4 (code remplacé)

E2 — Jeton de session stocké en clair

📍 MainActivity.kt, ligne 38

```
prefs.edit().putBoolean("logged_in", true).putString("session_token", token).apply()
```

❌ **Pourquoi c'est dangereux :** `MODE_PRIVATE` isole le fichier mais ne le chiffre pas : avec un accès debug (`run-as`), root ou une sauvegarde, le jeton se lit directement, et le voler suffit pour usurper la session.

✅ **Correction :** Nouveau composant `SecureTokenStore` : chiffrement **AES-GCM** (clé AES-256 générée dans **Android Keystore**, IV aléatoire de 12 octets, tag de 16 octets). Seul `[version][IV][chiffré+tag]` est stocké.

```
store.save(token)   // → secure_session.xml : encrypted_token = "Af2N1bcf…" (illisible)
```

Preuve dans ce rapport : Étapes 1 (avant), 3–4 (correction), 5 (contre-test)

E3 — Note privée stockée en clair

📍 MainActivity.kt, ligne 49

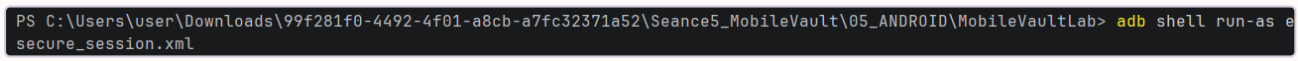
```
prefs.edit().putString("private_note", note.text.toString()).apply()
```

❌ **Pourquoi c'est dangereux :** Même problème que E2 : la donnée « privée » est lisible en clair dans `vault_lab.xml`.

✅ **Correction :** La version corrigée n'enregistre plus de note (si la fonctionnalité était conservée, elle serait chiffrée avec `SecureTokenStore`). **Migration :** au démarrage, l'application supprime l'ancien fichier `vault_lab.xml`, qui contenait encore la note et le jeton en clair. Après correction, `ls shared_prefs` ne montre plus que `secure_session.xml`.

```
store = SecureTokenStore(this)
deleteSharedPreferences("vault_lab")   // migration : supprime l'ancien stockage en clair
```

Preuve dans ce rapport : Étape 1 (avant), étape 4 (fonctionnalité retirée), capture de migration ci-dessous



Après migration : seul le stockage chiffré `secure_session.xml` reste sur l'appareil ; l'ancien fichier en clair a été supprimé.

E4 — Session représentée par un simple booléen

📍 MainActivity.kt, lignes 34 et 48–57

```
private fun isLoggedIn(): Boolean = prefs.getBoolean("logged_in", false)
...
if (!isLoggedIn()) { show("Accès refusé : session locale absente"); return }
```

❌ **Pourquoi c'est dangereux :** Le contrôle d'accès repose sur une valeur `true/false` modifiable dans le fichier : il suffit de la passer à `true` pour « être connecté ». Rien n'est vérifié.

✅ **Correction :** La session est maintenant dérivée du **déchiffrement réussi** du jeton (`store.load() != null`) : un jeton absent, altéré ou sans clé donne « Aucune session ». La **validité réelle** (expiration, révocation, droits) reste une décision du serveur.

```
val token = store.load()
status.text = if (token == null) "Aucune session (ou donnée invalide)" else "Session locale
restaurée"
```

Preuve dans ce rapport : Étapes 1 (avant), 4 (correction), 7 (expiration locale ≠ serveur)

E5 — Base64 présenté comme une protection

📍 VaultCrypto.kt, ligne 7 · bouton « Protéger note (fonction à auditer) »

```
fun protect(plain: String): String =
    Base64.encodeToString(plain.toByteArray(Charsets.UTF_8), Base64.NO_WRAP)
```

❌ **Pourquoi c'est dangereux :** Base64 est un **encodage** : réversible par tout le monde, sans clé (cours 3.1). La note « protégée » se décode en une seconde : c'est une fausse sécurité.

✅ **Correction :** Remplacé par un vrai chiffrement authentifié (AES-GCM + Keystore). Base64 n'est plus utilisé que pour écrire les octets chiffrés sous forme de texte. Le fichier `VaultCrypto.kt` (fausse protection) a été **supprimé du projet**.

```
val ciphertextAndTag = cipher.doFinal(token.toByteArray(Charsets.UTF_8))  // AES-GCM
val encoded = Base64.encodeToString(record, Base64.NO_WRAP)               // simple mise en
texte
```

Preuve dans ce rapport : Étape 3 (correction), étape 5 (Base64 ≠ chiffrement), question 1

E6 — Journalisation du jeton (log sensible)

📍 MainActivity.kt, ligne 39

```
Log.d(LOG_TAG, "Debug token=$token")
```

❌ **Pourquoi c'est dangereux :** Le jeton apparaît en clair dans Logcat, lisible par un PC en USB, des outils de debug ou des rapports de plantage, même si le stockage est chiffré.

✅ **Correction :** Log supprimé (TP 1), puis remplacé par un **événement neutre** sans identifiant de session (TP 2).

```
Log.d("MobileVaultLab", "Connexion fictive réussie (session enregistrée)")
```

Preuve dans ce rapport : Étape 2 (avant), étape 5 (contre-test : Logcat vide)

E7 — Sauvegarde permissive

📍 AndroidManifest.xml, ligne 3 · data_extraction_rules.xml, lignes 3–4

```
android:allowBackup="true"
<cloud-backup><include domain="sharedpref" path="."/></cloud-backup>
<device-transfer><include domain="sharedpref" path="."/></device-transfer>
```

❌ **Pourquoi c'est dangereux :** Toutes les préférences (jeton, note) partent dans les sauvegardes cloud et les transferts vers un autre appareil. Sans sa clé Keystore (qui ne quitte jamais l'appareil), un jeton chiffré restauré est inutilisable ; un jeton en clair, lui, est exposé.

✅ **Correction :** `allowBackup="false"` (versions anciennes) et exclusion des fichiers sensibles dans `data_extraction_rules.xml` (Android 12+).

```
android:allowBackup="false"
<cloud-backup><exclude domain="sharedpref" path="secure_session.xml"/> …</cloud-backup>
<device-transfer><exclude domain="sharedpref" path="secure_session.xml"/> …</device-transfer>
```

Preuve dans ce rapport : Signalé au TP 1 (question 5), corrigé au TP 2 (P4)

Étape 0 — Préparer l'environnement

But : vérifier que tout fonctionne avant de chercher des failles.

1. Ouvrir dans Android Studio le dossier **MobileVaultLab** (celui qui contient `settings.gradle.kts`), et non le dossier parent. Sinon, Android Studio ne reconnaît pas le projet et le bouton Run reste grisé.
2. Attendre la synchronisation Gradle. Ne pas lancer l'« AGP Upgrade Assistant » : le projet est prévu pour la version 8.7.3.

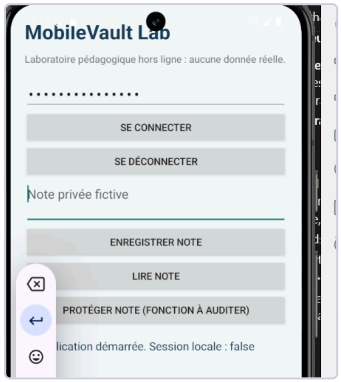
3. Créer et démarrer un émulateur (Pixel 8, API 35), puis cliquer sur **Run ▶**.
4. Dans le Terminal (PowerShell), ajouter `adb` au PATH puis vérifier l'émulateur :

```
$env:Path += ";$env:LOCALAPPDATA\Android\Sdk\platform-tools"
adb devices
```

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID> adb devices
List of devices attached
emulator-5554   device

PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID>
```

L'émulateur est détecté avec l'état « device ».



L'application MobileVault démarre : un champ mot de passe, des boutons de connexion et de déconnexion, un champ de note et trois boutons de gestion de la note.

> **Difficultés rencontrées :** `adb` n'était pas reconnu (il n'est pas dans le PATH de Windows), et le projet avait d'abord été ouvert un niveau trop haut. Ces deux points ont été corrigés comme indiqué ci-dessus.

> La commande du PATH ne vaut que pour le terminal en cours : il faut la retaper dans chaque nouveau terminal.

Étape 1 — Créer une session et observer le stockage

But : voir de ses propres yeux ce que l'application écrit sur le téléphone.

1. Taper le mot de passe `vault-demo-2026` → **Se connecter**.
2. Écrire une note fictive → **Enregistrer note**.
3. Lire le fichier de préférences de l'application :

```
adb shell run-as edu.lab.mobilevault cat shared_prefs/vault_lab.xml
```

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab>  adb shell run-as edu.lab.mobilevault cat shared
_prefs/vault_lab.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="session_token">DEMO_SESSION_TOKEN_2026</string>
    <boolean name="logged_in" value="true" />
    <string name="private_note">Ma note secrète de test</string>
</map>
```

Capture 1 — Le jeton, l'état de connexion et la note sont lisibles en clair.

Dans le code (`MainActivity.kt`, fonction `login()`) :

```
prefs.edit().putBoolean("logged_in", true).putString("session_token", token).apply()
```

Explication :

- `session_token` est en clair : le voler suffit pour usurper la session.
- `logged_in = true` n'est qu'un booléen : il suffirait de le modifier pour paraître connecté, car l'application ne vérifie rien auprès d'un serveur.
- La note privée est elle aussi en clair.
- Le mot de passe `vault-demo-2026` est écrit en dur dans le code (`LAB_PASSWORD`) : n'importe qui peut le lire en décompilant l'APK.

> **Pourquoi `MODE_PRIVATE` n'est pas un chiffrement :** il empêche seulement les *autres applications* de lire le fichier. Le contenu reste en clair pour quiconque a un accès debug (`run-as`), root ou une sauvegarde. `run-as` fonctionne ici parce que l'application est en mode *debug*.

Étape 2 — Observer la fuite dans Logcat

But : montrer qu'un secret peut fuir ailleurs que dans le stockage.

1. Vider les anciens journaux : `adb logcat -c`

2. Cliquer de nouveau sur **Se connecter**.
3. Lire les journaux de l'application :

```
adb logcat -d -s MobileVaultLab:D
```



Capture 2 — Le jeton apparaît en clair dans les journaux.

Dans le code : `Log.d(LOG_TAG, "Debug token=$token")`.

Explication : `-d` affiche les journaux puis s'arrête ; `-s MobileVaultLab:D` filtre sur le tag de l'application au niveau Debug. Les journaux sont lisibles par un PC branché en USB, des outils de debug ou des rapports de plantage. Le jeton sort donc par **deux endroits** : le fichier XML et Logcat.

> Les lignes de code Kotlin ne se tapent pas dans le Terminal : le Terminal ne reçoit que des commandes (`adb …`).

Étape 3 — Créer le composant de stockage chiffré

But : remplacer le stockage en clair par un composant **isolé**, qui est le seul à faire de la cryptographie.

Action : clic droit sur le package `edu.lab.mobilevault` → New → Kotlin Class/File → **SecureTokenStore**, puis coller le code fourni (en adaptant la ligne `package`).

3.1 Création de la clé dans le Keystore

```
val generator = KeyGenerator.getInstance(KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore")
KeyGenParameterSpec.Builder(alias, PURPOSE_ENCRYPT or PURPOSE_DECRYPT)
    .setKeySize(256).setBlockModes(BLOCK_MODE_GCM)
    .setEncryptionPaddings(ENCRYPTION_PADDING_NONE)
```

La clé AES-256 est **générée directement dans le Keystore** sous un **alias**. L'application ne voit jamais la clé : elle ne garde que son nom. Même en lisant la mémoire ou les fichiers de l'application, on ne la trouve pas.

3.2 Chiffrement : `save()`

```
cipher.init(Cipher.ENCRYPT_MODE, key)
val iv = cipher.iv                        // nonce de 12 octets, nouveau à chaque fois
val ciphertextAndTag = cipher.doFinal(token.toByteArray())
val record = byteArrayOf(1) + iv + ciphertextAndTag   // [version][IV][chiffré+tag]
```

- Le **nonce/IV** est tiré au hasard à chaque chiffrement, donc le même jeton donne un chiffré différent à chaque fois. Il n'est pas secret et il est stocké à côté du chiffré.
- AES-GCM produit le **texte chiffré** et un **tag d'authentification** de 16 octets, qui sert de « sceau » contre les modifications.
- L'ensemble est encodé en **Base64**, uniquement pour pouvoir l'écrire comme du texte dans le XML.

3.3 Déchiffrement : `load()`

La fonction relit la version et l'IV, puis déchiffre avec `GCMParameterSpec(128, iv)`. `doFinal()` **vérifie le tag**. En cas d'erreur (donnée altérée, clé absente, format inconnu), l'enregistrement est effacé (`clear()`) et la fonction renvoie `null` : l'utilisateur doit se reconnecter.

3.4 Fonctions de test

`tamperForLab()` inverse volontairement un bit du chiffré, pour tester l'étape 6.
`encryptedRecordForLab()` sert seulement à observer le chiffré. Ces deux fonctions **ne doivent pas exister en production**.

Étape 4 — Brancher le coffre à l'interface

But : un composant de sécurité qui n'est pas utilisé ne protège rien. Il faut remplacer le chemin vulnérable dans l'application.



Action : remplacer tout le contenu de `MainActivity.kt` par la version corrigée, avec `package edu.lab.mobilevault`, puis Build → Rebuild Project et Run ▶.

Ce qui change :

- l'Activity utilise `SecureTokenStore` (`save`, `load`, `clear`) ;
- plus aucun `Log.d` du jeton ;
- le jeton **n'est jamais affiché** : l'interface ne reçoit que l'état de la session ;
- cinq boutons de test : connexion fictive, restaurer, altérer, expiration locale, déconnexion.

> **Erreur rencontrée :** le code du guide commençait par `package edu.lab.securebank`. Le build échouait et l'ancienne interface restait affichée. Il a fallu mettre `edu.lab.mobilevault` dans les deux fichiers.

Résultat : après « 1. Connexion fictive » puis « 2. Restaurer session », l'écran affiche « Session locale restaurée ».

Étape 5 — Prouver que le jeton n'est plus en clair

But : ne pas se contenter de lire le code, mais vérifier ce qui est réellement écrit sur le téléphone.

```
adb shell run-as edu.lab.mobilevault cat shared_prefs/secure_session.xml
```

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb shell run-as edu.lab.mobilevault cat shared_prefs/secure_session.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="encrypted_token">AZAR8WSNk6c4EJkL2/uzGf9Y8IQM6rLrAMEIoj/sROvzuIOkdpm2QfndT5pUBAGNngKSZ2csNuTwqpEbrmbEgKT9LmTpNg==</string>
</map>
```

Capture 3 — Seul `encrypted_token` est stocké : une chaîne illisible, sans `demo_`.

Le premier caractère `A` du Base64 correspond à l'octet de version (1). Viennent ensuite l'IV, puis le chiffré et le tag. Le Base64 ne protège rien : c'est AES-GCM qui rend la donnée illisible sans la clé.

```
adb logcat -c      (puis « 1. Connexion fictive »)
adb logcat -d -s MobileVaultLab:D
```

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb logcat -c
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb logcat -d -s MobileVaultLab:D
```

Logcat après correction : aucune ligne, donc plus de fuite du jeton.

> Attention aux fautes de frappe : avec un tag mal écrit (par exemple `MobileVaitLab`), la commande ne renvoie rien non plus, et le test ne prouve rien.

Étape 6 — Vérifier l'intégrité (AES-GCM)

But : vérifier qu'une donnée modifiée n'est **jamais acceptée en silence** (cours 3.4).

Action : 1. Connexion fictive → 3. Altérer le chiffré → 2. Restaurer session, puis relire le fichier.




```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab>  adb shell run-as edu.lab.mobilevault cat shared_prefs/secure_session.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map />
```

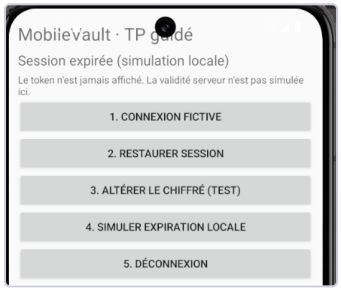
Capture 4 — « Aucune session (ou donnée invalide) » et fichier vide (`<map />`).

Explication : `tamperForLab()` inverse un bit (`record[13] xor 1`). Au chargement, AES-GCM recalcule le tag et le compare au tag stocké. Comme ils sont différents, `doFinal()` lève une exception, et `load()` efface l'enregistrement puis renvoie `null`.

> **À retenir :** c'est **tout ou rien**. Le déchiffrement s'arrête sur une erreur, il ne rend jamais une donnée à moitié fausse. On est protégé contre les modifications silencieuses, qu'elles viennent d'une corruption ou d'une attaque.

Étape 7 — Déconnexion et expiration locale

But : distinguer la suppression du jeton sur l'appareil et la validité réelle côté serveur.



Action : 1. Connexion fictive → 4. Simuler expiration locale (écran ci-contre : « Session expirée ») → 5. Déconnexion, puis relire le fichier :

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb shell run-as edu.lab.mobilevault cat shared_prefs/secure_session.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map />
```

Après la déconnexion, le fichier est vide : le jeton chiffré a été effacé.

Notion	Ce que c'est
Expiration locale	Simple variable en mémoire (`expiresAt`). Le jeton est toujours stocké et n'est révoqué nulle part.
Expiration serveur	Le serveur décide qu'un jeton n'est plus valide après un certain temps. C'est la seule qui fait foi.
Révocation	Le serveur annule un jeton avant son expiration (déconnexion, vol du téléphone).
Déconnexion locale	Efface le jeton du téléphone. Une vraie déconnexion doit aussi demander la révocation au serveur.

Étape 8 — Bilan et réponses aux questions

Comparaison avant / après

	Avant correction	Après correction
Fichier de préférences	Jeton + note en clair (`vault_lab.xml`)	Jeton chiffré AES-GCM (`secure_session.xml`)
Clé de chiffrement	Aucune (Base64 présenté comme protection)	AES-256 dans Android Keystore, jamais exposée
Logcat	`Debug token=DEMO_SESSION_TOKEN_2026`	Aucune fuite
Donnée modifiée	Acceptée sans contrôle	Refusée grâce au tag GCM
Affichage	—	Le jeton n'est jamais affiché

Réponses aux questions

1. Pourquoi Base64 n'est-il pas une protection ?
Base64 est un **encodage** : il change seulement la représentation des données, il est réversible par tout le monde et n'utilise pas de clé. Il sert seulement à écrire des octets sous forme de texte.

2. Que contient le fichier secure_session.xml ?
Une seule entrée, `encrypted_token`, qui contient en Base64 : 1 octet de version, l'IV (12 octets), puis le texte chiffré et le tag (16 octets). Ni le jeton en clair ni la clé.

3. Pourquoi une clé Keystore peut-elle devenir indisponible ?
Elle disparaît à la désinstallation ou à l'effacement des données de l'application. Elle peut aussi être invalidée par un changement du verrouillage de l'écran ou de la biométrie (si elle y est liée), et elle n'existe pas après une restauration sur un autre appareil, car elle ne quitte jamais le Keystore. Dans ce cas, le déchiffrement échoue et l'utilisateur doit se reconnecter.

4. Pourquoi le contrôle d'autorisation doit-il rester côté serveur ?
Tout ce qui est sur le téléphone (un booléen `logged_in`, une date d'expiration locale, le code de l'application) peut être lu ou modifié par un attaquant. Seul le serveur peut vérifier qu'un jeton est valide, non expiré et non révoqué.

5. Quelles méthodes ne doivent pas être conservées en production ?
`tamperForLab()`, `encryptedRecordForLab()`, le bouton « Altérer le chiffré » et l'expiration simulée. Il faut aussi supprimer les défauts de l'ancien code : mot de passe en dur, `Log.d` du jeton, `VaultCrypto` en Base64 (fichier supprimé du projet), et la sauvegarde permissive (`allowBackup="true"` avec des règles qui incluent les SharedPreferences).

Récapitulatif : vulnérabilités → corrections

N°	Vulnérabilité	Statut	Où c'est prouvé
E1	Secret (mot de passe) écrit en dur dans le code	**Corrigé**	Étape 1 (découverte dans le code), étape 4 (code remplacé)
E2	Jeton de session stocké en clair	**Corrigé**	Étapes 1 (avant), 3–4 (correction), 5 (contre-test)
E3	Note privée stockée en clair	**Corrigé**	Étape 1 (avant), étape 4 (fonctionnalité retirée), capture de migration ci-dessous
E4	Session représentée par un simple booléen	**Corrigé**	Étapes 1 (avant), 4 (correction), 7 (expiration locale ≠ serveur)
E5	Base64 présenté comme une protection	**Corrigé**	Étape 3 (correction), étape 5 (Base64 ≠ chiffrement), question 1
E6	Journalisation du jeton (log sensible)	**Corrigé**	Étape 2 (avant), étape 5 (contre-test : Logcat vide)
E7	Sauvegarde permissive	**Corrigé au TP 2**	Signalé au TP 1 (question 5), corrigé au TP 2 (P4)

Toutes les vulnérabilités identifiées sont corrigées et contre-testées. E7 (sauvegarde) a été signalée au TP 1 puis corrigée au TP 2 (P4).

> **Limites de la solution :** le Keystore protège la **clé** mais pas son **usage**. Sur un appareil compromis, un attaquant peut faire déchiffrer des données par le Keystore pendant que l'application tourne. La protection matérielle (TEE, StrongBox) augmente le coût d'une attaque sans la rendre impossible. La sécurité finale repose toujours sur le serveur.

> **Conclusion :** le chemin de preuve est complet. Vulnérabilité observée (captures 1 et 2) → correction (AES-GCM + Keystore, composant isolé) → contre-test (capture 3 et Logcat vide) → test d'intégrité (capture 4) → limites identifiées.